

Embedded System Design and Implementation Report

Multiplexed 7-Segment Clock and Environmental Monitoring System Based on ATmega32 Microcontroller

Author:	Mohsen Safari
Faculty:	Faculty of Electrical and Computer Engineering
Institution:	Shahid Beheshti University
Microcontroller:	ATmega32 @ 8 MHz
Display:	7SEG-MPX6-CA (6-digit common-anode)
Tools:	CodeVisionAVR, Proteus Design Suite

July 20, 2026

Abstract

This report presents the design and implementation of an embedded system based on the Atmel ATmega32 8-bit microcontroller. The system drives a multiplexed six-digit common-anode seven-segment display (7SEG-MPX6-CA) that alternates between four screens: a 24-hour clock in HH:MM:SS format with two blinking colons, a Celsius temperature screen, a Fahrenheit temperature screen, and a relative-humidity screen. Environmental readings are acquired from a companion sensor board through a 9600 baud UART link, in which the ATmega32 transmits a single request byte ('C', 'F' or 'H') and receives one byte carrying the measured value already scaled to the requested unit. Two external interrupts (INT0 and INT1) and a four-key keypad on Port A allow the user to cycle between screens, enter an edit mode, and adjust the hours, minutes, or seconds fields. While editing, the digit pair under the cursor blinks at approximately 2 Hz to indicate the field being changed, while the clock continues to run in the background. Timing is split across two hardware timers: Timer1 generates a precise one-second interrupt for timekeeping, while Timer2 runs at a 2 ms cadence for display multiplexing, keypad debouncing, edit-cursor blinking, and heartbeat LED toggling. The firmware is written in C for CodeVisionAVR and validated through Proteus simulation. The results confirm correct multiplexing, screen cycling, edit-mode blinking of the selected digit pair, and acquisition of environmental data over UART.

Contents

Abstract	ii
Abbreviations	vii
1 Introduction	1
1.1 Project Overview	1
1.2 Objectives	1
1.3 System Requirements	2
1.4 Report Organization	2
2 System Architecture	3
2.1 Pin Mapping	3
2.2 Required Components	3
2.3 Seven-Segment Multiplexing Principle	4
2.4 Overall System Block Diagram	5
2.5 System Operation	5
2.6 Firmware Architecture	6
3 Hardware Design	8
3.1 Microcontroller Description	8
3.2 Circuit Schematic	9
4 Software Design	10
4.1 Program Structure	10
4.2 Timer Configuration	10
4.3 Peripheral Configuration	11
4.4 Interrupt Architecture	11
4.4.1 INT0 — Screen Cycle	11
4.4.2 INT1 — Edit Mode Toggle	12
4.4.3 TIM1_COMPA — One-Second Timekeeping	12
4.4.4 TIM2_COMP — 2 ms Multiplexing Heartbeat	12
4.4.5 USART_RXC — Sensor Data Reception	12
4.5 Display Buffer Mechanism	12
4.6 Keypad Debouncing	13
4.7 Global Variables	13
5 Implementation	14
5.1 Development Environment	14
5.2 Main Program	14
5.3 Key Implementation Details	14
5.3.1 Timer1 Compare ISR (One-Second Timekeeping)	14
5.3.2 Timer2 Compare ISR (2 ms Multiplexing Heartbeat)	15

5.3.3	Edit-Cursor Blink Mapping	15
5.3.4	Environmental Value Formatting	16
5.3.5	Keypad Editing of Hours/Minutes/Seconds	17
6	Timing Analysis	18
6.1	Timer1 Timing Diagram	18
6.2	Timer2 Multiplexing Timing	19
6.3	UART Communication Timing	19
7	Testing and Results	20
7.1	Simulation Environment	20
7.2	Simulation Results	20
7.3	UART Manual Reception Verification	21
7.3.1	Celsius Mode	21
7.3.2	Fahrenheit Mode	22
7.3.3	Humidity Mode	22
7.3.4	Negative Value Handling	23
7.3.5	Direct Variable Inspection	24
7.4	Experimental Results	25
7.5	Debugging and Design Iterations	25
8	Conclusion and Future Work	26
8.1	Conclusion	26
8.2	Future Work	26
	Source Code Availability	27
	References	28
A	Firmware Listing Reference	29

List of Figures

2.1	System block diagram showing the ATmega32 controller board, the 7SEG-MPX6-CA display, the keypad, control push-buttons, heartbeat LED, and the remote sensor board communicating over UART.	5
2.2	System finite state machine: four display modes cycled by INT0, edit-mode toggled by INT1, cursor sub-states, and key actions.	6
2.3	Firmware architecture: main loop managing display update and temperature polling, with five interrupt service routines handling screen switching, edit-mode toggling, timekeeping, multiplexing, and UART reception.	7
3.1	Complete circuit schematic of the controller board.	9
6.1	Timing Diagram	18
7.1	Proteus simulation screenshot showing the ATmega32 driving the 7SEG-MPX6-CA display.	20
7.2	Manual UART test in Celsius mode: byte 0x19 (25 decimal) sent in response to the 'C' request.	21
7.3	Manual UART test in Fahrenheit mode: byte 0x4D (77 decimal) sent in response to the 'F' request.	22
7.4	Manual UART test in Humidity mode: byte 0x3C (60 decimal) sent in response to the 'H' request.	22
7.5	Manual UART test with a negative reading: byte 0xFB (two's-complement representation of -5) sent while in Humidity mode.	23
7.6	Proteus Variables window confirming <code>current_temp</code> holds the exact byte received (0x19), with <code>display_buf</code> correctly populated as 0x0A, 0x02, 0x05, 0x0A, 0x0A, 0x0C.	24

List of Tables

2.1	ATmega32 pin mapping	3
2.2	Hardware components	4
4.1	Interrupt vector table and priority	11
4.2	Global variable declarations and purpose	13
7.1	Summary of manual UART injection test cases	24
7.2	Summary of functional test cases and observed results	25

Abbreviations

Abbreviation	Meaning
AVR	Advanced Virtual RISC (Atmel microcontroller family)
CTC	Clear Timer on Compare match
FSM	Finite State Machine
ISR	Interrupt Service Routine
UART	Universal Asynchronous Receiver/Transmitter
UBRR	USART Baud Rate Register
8N1	8 data bits, No parity, 1 stop bit

Chapter 1

Introduction

1.1 Project Overview

The project implements a compact embedded instrument that combines a real-time clock with a remote environmental monitoring capability on a single ATmega32 microcontroller. A six-digit common-anode seven-segment display (7SEG-MPX6-CA) is time-multiplexed so that the same six physical digits are reused to present four logically different screens: the current time in HH:MM:SS format, the temperature in degrees Celsius, the temperature in degrees Fahrenheit, and the relative humidity in percent. The user selects the active screen through a push-button connected to external interrupt INT0, while a second push-button on INT1 toggles a clock-edit mode in which a four-key keypad allows the hours, minutes, and seconds to be incremented or decremented.

Environmental readings are not measured directly on the ATmega32 board. A separate sensor microcontroller, programmed with `SysDig2AVRProject.hex`, performs the analog measurements and exposes them through a 9600 baud, 8N1 UART link. The ATmega32 sends a one-byte request identifying the desired unit and receives a single-byte reply already expressed in that unit, keeping the main firmware free of unit-conversion arithmetic.

1.2 Objectives

The principal objectives of the project are:

- Implement flicker-free multiplexing of a six-digit seven-segment display using a timer-driven interrupt service routine.
- Maintain a software 24-hour clock with second, minute, and hour counting using Timer1 in CTC mode.
- Provide a user interface with two external interrupts and a four-key keypad for screen selection and time editing of hours, minutes, and seconds.
- Visualize the field being edited by blinking the corresponding digit pair at a comfortable rate, while the clock keeps running.
- Acquire remote temperature and humidity data over UART on demand, once per minute.
- Demonstrate the design through Proteus simulation and produce CodeVisionAVR-compilable firmware.

1.3 System Requirements

Functional requirements:

- Display four screens sequentially, selectable by INT0: TIME, TEMP, FAHR, HUM.
- Keep a 24-hour clock (0–23 hours) in HH:MM:SS format, with the two colons blinking once per second.
- Allow editing of hours, minutes, and seconds. The cursor moves between these three fields using LEFT/RIGHT; UP/DOWN increment or decrement the active field with wrap-around.
- Poll the sensor board once per minute for the unit corresponding to the currently displayed screen.
- Display signed Celsius and Fahrenheit values (including a minus sign) and unsigned humidity (0–100%), with the unit letter shown in the last digit and the value right-justified in digits 0–2 (digits 3–4 blank for humidity; for C/F modes a separator glyph appears in digit 4).

Non-functional requirements:

- System clock frequency: 8 MHz.
- UART baud rate: 9600 baud, 8N1.
- Multiplexing refresh rate: one digit every 2 ms (500 Hz per digit, ≈ 83 Hz full-frame refresh over six digits).
- Edit-cursor blink rate: approximately 2 Hz.
- Compilable with CodeVisionAVR and simulatable in Proteus.

1.4 Report Organization

Chapter 2 presents the system architecture, including the pin assignment, the multiplexing principle, and the overall operating behavior. Chapter 3 describes the hardware design. Chapter 4 details the software design, including timer configuration, interrupt architecture, and the display buffer mechanism. Chapter 5 presents implementation details together with the key source-code listings. Chapter 6 provides a timing analysis of the system, Chapter 7 reports the simulation and experimental results, and Chapter 8 concludes the report and outlines future improvements.

Chapter 2

System Architecture

2.1 Pin Mapping

Table 2.1 provides the complete pin assignment for all I/O ports of the ATmega32.

Table 2.1: ATmega32 pin mapping

Port	Pin	Function	Connection
A	PA0	Keypad UP	Keypad row 1 (internal pull-up)
	PA1	Keypad DOWN	Keypad row 2 (internal pull-up)
	PA2	Keypad RIGHT	Keypad row 3 (internal pull-up)
	PA3	Keypad LEFT	Keypad row 4 (internal pull-up)
B	PB0	Segment a	7-segment line a via 220 Ω
	PB1	Segment b	7-segment line b via 220 Ω
	PB2	Segment c	7-segment line c via 220 Ω
	PB3	Segment d	7-segment line d via 220 Ω
	PB4	Segment e	7-segment line e via 220 Ω
	PB5	Segment f	7-segment line f via 220 Ω
	PB6	Segment g	7-segment line g via 220 Ω
	PB7	Segment dp	Decimal point via 220 Ω
C	PC0	Digit enable 1	7SEG-MPX6-CA common-anode DIG1
	PC1	Digit enable 2	7SEG-MPX6-CA common-anode DIG2
	PC2	Digit enable 3	7SEG-MPX6-CA common-anode DIG3
	PC3	Digit enable 4	7SEG-MPX6-CA common-anode DIG4
	PC4	Digit enable 5	7SEG-MPX6-CA common-anode DIG5
	PC5	Digit enable 6	7SEG-MPX6-CA common-anode DIG6
D	PD0	TXD	UART transmit to sensor board
	PD1	RXD	UART receive from sensor board
	PD2	INT0 (falling edge)	Screen-cycle push-button
	PD3	INT1 (falling edge)	Edit-mode push-button
	PD4	Output	Heartbeat LED (1 Hz toggle)
	PD5–PD7	Unused	Left unconnected

2.2 Required Components

Table 2.2 lists all components required to build the system.

Table 2.2: Hardware components

Component	Qty	Specification / Function
ATmega32 microcontroller	1	8-bit AVR, 32 KB flash, 2 KB SRAM, 8 MHz
7SEG-MPX6-CA display	1	6-digit common-anode, multiplexed seven-segment
Sensor board	1	Separate AVR running <code>SysDig2AVRProject.hex</code>
Push-buttons (momentary)	2	Falling-edge, with internal pull-ups (INT0, INT1)
Keypad (4-key)	1	Active-low, internal pull-ups (UP/DOWN/LEFT/RIGHT)
LED (5 mm)	1	Heartbeat indicator, PD4 output
Resistor (220 Ω)	8	Current limiting for segment lines a–dp
Resistor (470 Ω)	1	Current limiting for heartbeat LED
Crystal (8 MHz)	1	System clock source
Ceramic capacitor (22 pF)	2	Crystal load capacitors
Capacitor (100 nF)	2	Decoupling capacitors for IC
Voltage regulator (7805)	1	5 V regulation for board supply
Power supply (9–12 V DC)	1	Input to voltage regulator
Breadboard / PCB	1	Prototyping board
Connecting wires	As needed	Jumper wires for interconnections

2.3 Seven-Segment Multiplexing Principle

The 7SEG-MPX6-CA is a six-digit common-anode seven-segment display. In a common-anode configuration, all segment anodes within a single digit are connected together to form the common-anode pin, while the cathodes of each segment are independent. A digit is enabled by driving its common-anode pin HIGH, and a segment illuminates when its cathode is pulled LOW.

Multiplexing reduces the number of I/O pins required. Instead of dedicating 8 pins per digit (48 pins total), the segment lines of all six digits are connected in parallel, and only one digit is enabled at any given time. By rapidly switching between digits at a rate exceeding the persistence of vision (typically >60 Hz), the human eye perceives all six digits as continuously lit.

The multiplexing timing is as follows:

- Each digit is activated for 2 ms (driven by the Timer2 compare-match ISR).
- A complete frame (all six digits) takes $6 \times 2 \text{ ms} = 12 \text{ ms}$.
- The frame refresh rate is $1/12 \text{ ms} \approx 83.3 \text{ Hz}$, well above the flicker-fusion threshold.

The multiplexing sequence is:

1. All digits are turned off (`PORTC = 0x00`) to prevent ghosting during transition.
2. The segment pattern for the current digit is written to `PORTB`.
3. The colon decimal points (for TIME mode) are conditionally illuminated on digits 2 and 4.
4. The current digit is enabled by setting the corresponding bit in `PORTC`.
5. The digit counter advances to the next digit (modulo 6).

The segment mapping for common-anode operation (active-low) is defined in the `seg_code[]` table. For example, digit '0' is encoded as `0xC0` (binary 11000000), where bits a through dp map to PB0 through PB7 respectively, and a '0' bit means the segment is ON.

2.4 Overall System Block Diagram

The system is partitioned into two cooperating subsystems: the display/controller board (the subject of this report) and the remote sensor board (treated as a black box exposing a UART request/response interface). Figure 2.1 illustrates the high-level block diagram.

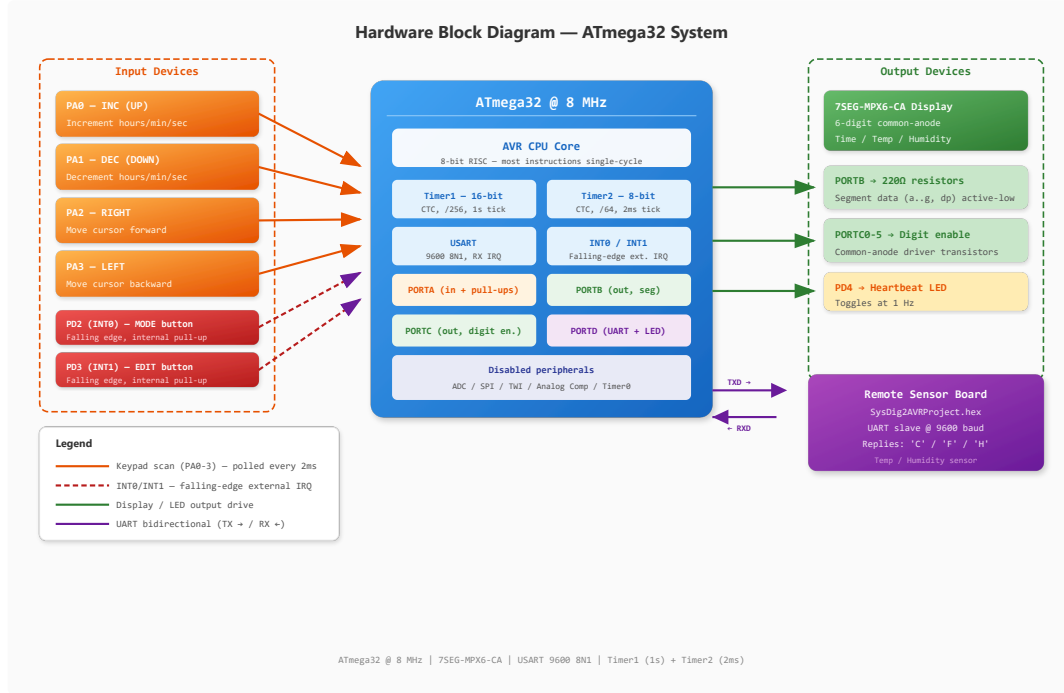


Figure 2.1: System block diagram showing the ATmega32 controller board, the 7SEG-MPX6-CA display, the keypad, control push-buttons, heartbeat LED, and the remote sensor board communicating over UART.

The ATmega32 sits at the centre of the controller board. Port B drives the seven-segment segment lines; Port C drives the six common-anode digit-enable lines (PC0-PC5); Port A hosts the four keypad inputs with internal pull-ups; Port D carries the UART TX/RX lines, the INT0/INT1 push-buttons, and a heartbeat LED on PD4. Communication with the sensor board is a single-wire-each asynchronous UART link.

2.5 System Operation

At power-up the system enters the TIME screen, showing the initial time 12:34:00. Each press of the INT0 button advances the active screen in the cyclic order

$$\text{TIME} \rightarrow \text{TEMP}_C \rightarrow \text{TEMP}_F \rightarrow \text{HUM} \rightarrow \text{TIME}.$$

On the TIME screen, pressing INT1 toggles the edit mode. Inside edit mode the keypad keys behave as follows: RIGHT moves the cursor to the next field (hours → minutes → seconds); LEFT moves it to the previous field; UP increments the field under the cursor with wrap-around (hours wrap at 24, minutes and seconds wrap at 60); DOWN decrements it. The clock continues to run during editing. The pair of digits of the field being edited blinks at approximately 2 Hz

so the user can clearly see which field is currently selected — this is the edit cursor, and is independent of the colon blink.

Whenever the active screen is one of the three sensor screens, the firmware arranges for a fresh reading to be requested once per minute. It transmits the unit byte ('C', 'F', or 'H') matching the displayed screen, and the USART receive interrupt stores the inbound byte in the shared `current_temp` variable. On sensor screens the timekeeping still runs in the background, so switching back to the TIME screen shows a correctly advanced clock.

A heartbeat LED on PD4 is toggled once per second by the Timer2 ISR, giving a visible confirmation that the firmware is alive.

Figure 2.2 shows the complete finite state machine governing screen transitions, edit-mode entry, and cursor navigation.

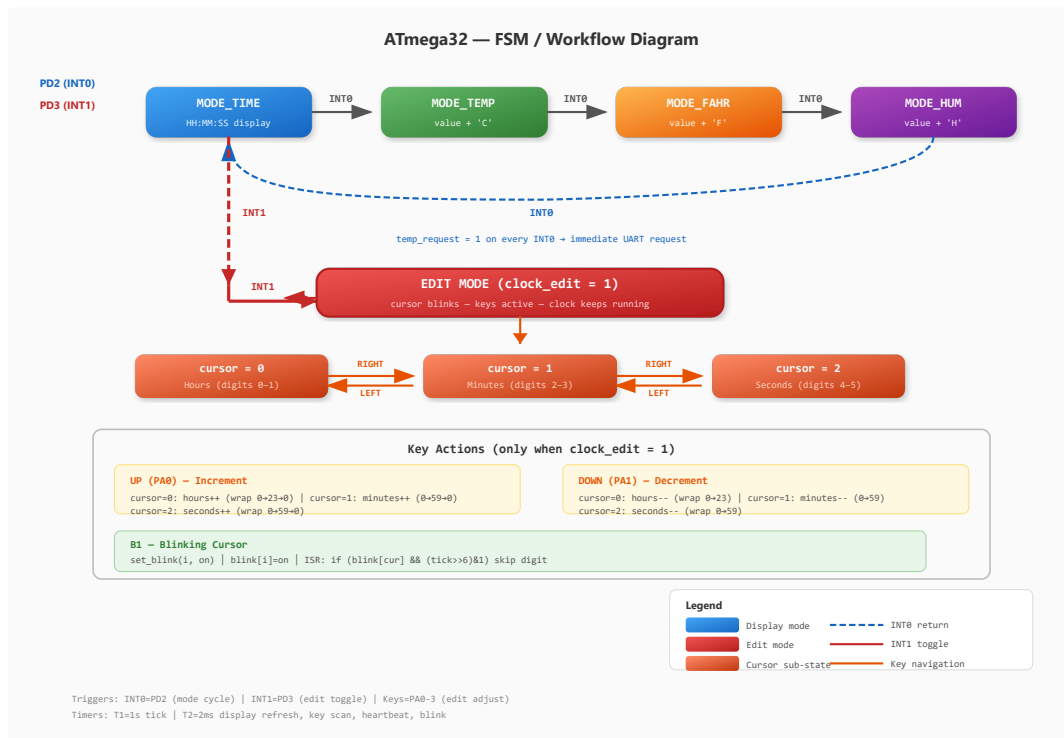


Figure 2.2: System finite state machine: four display modes cycled by INT0, edit-mode toggled by INT1, cursor sub-states, and key actions.

2.6 Firmware Architecture

The firmware is structured around a main super-loop and five interrupt service routines. Figure 2.3 shows the layered architecture: the main loop coordinates screen updates and sensor polling, while the Timer1, Timer2, INT0, INT1, and USART ISRs handle time-critical operations asynchronously.

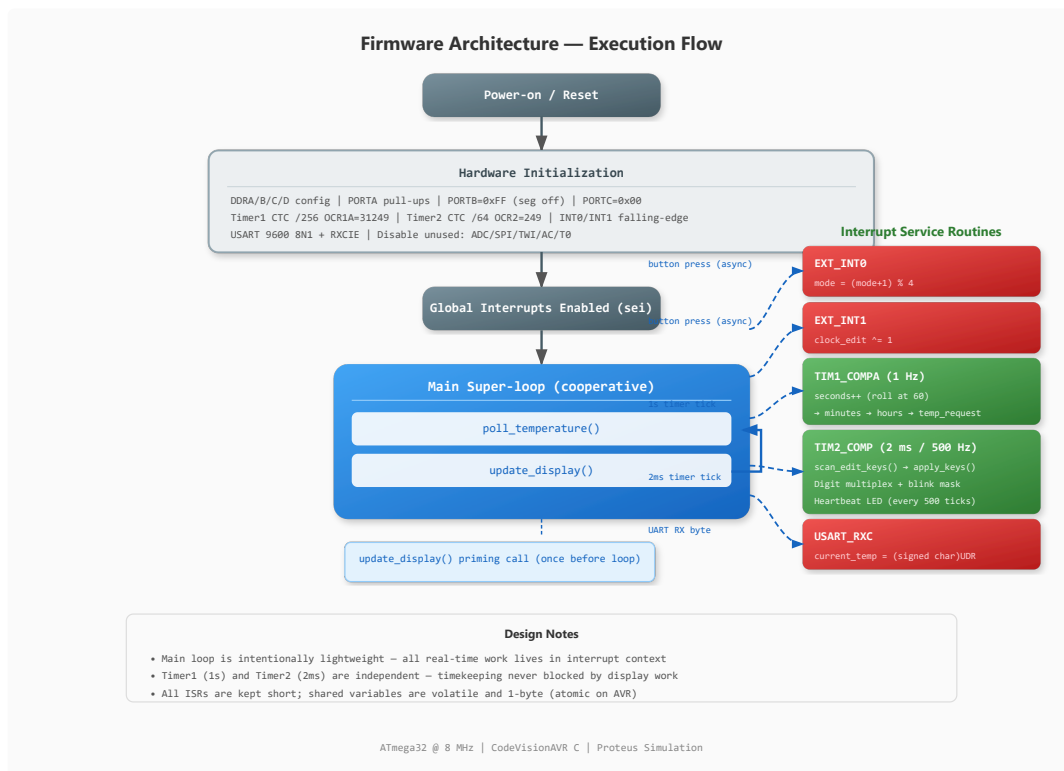


Figure 2.3: Firmware architecture: main loop managing display update and temperature polling, with five interrupt service routines handling screen switching, edit-mode toggling, timekeeping, multiplexing, and UART reception.

Chapter 3

Hardware Design

3.1 Microcontroller Description

The ATmega32 is an 8-bit AVR RISC microcontroller based on the enhanced AVR architecture, executing most instructions in a single cycle at clock rates up to 16 MHz. In this project it is operated at 8 MHz, which is sufficient for the multiplexing and UART tasks while keeping power consumption low. The device provides 32 KB of in-system programmable flash, 2 KB of SRAM, 1 KB of EEPROM, four 8-bit I/O ports (A–D), three timers (two 8-bit and one 16-bit), a USART, an SPI port, a TWI port, a 10-bit ADC, an analogue comparator, and four external interrupt sources.

The relevant peripherals for this design are: Timer1 in CTC mode for the one-second time-keeping base; Timer2 in CTC mode for the 2 ms display-multiplexing cadence; the USART for asynchronous communication with the sensor board; external interrupts INT0 and INT1 for the two control push-buttons; and Ports A, B, C, and D for the keypad, segment lines, digit-enable lines, and UART/button/LED lines respectively. Unused peripherals (ADC, analogue comparator, SPI, TWI, Timer0) are explicitly disabled at start-up to reduce power consumption and prevent spurious interrupts.

3.2 Circuit Schematic

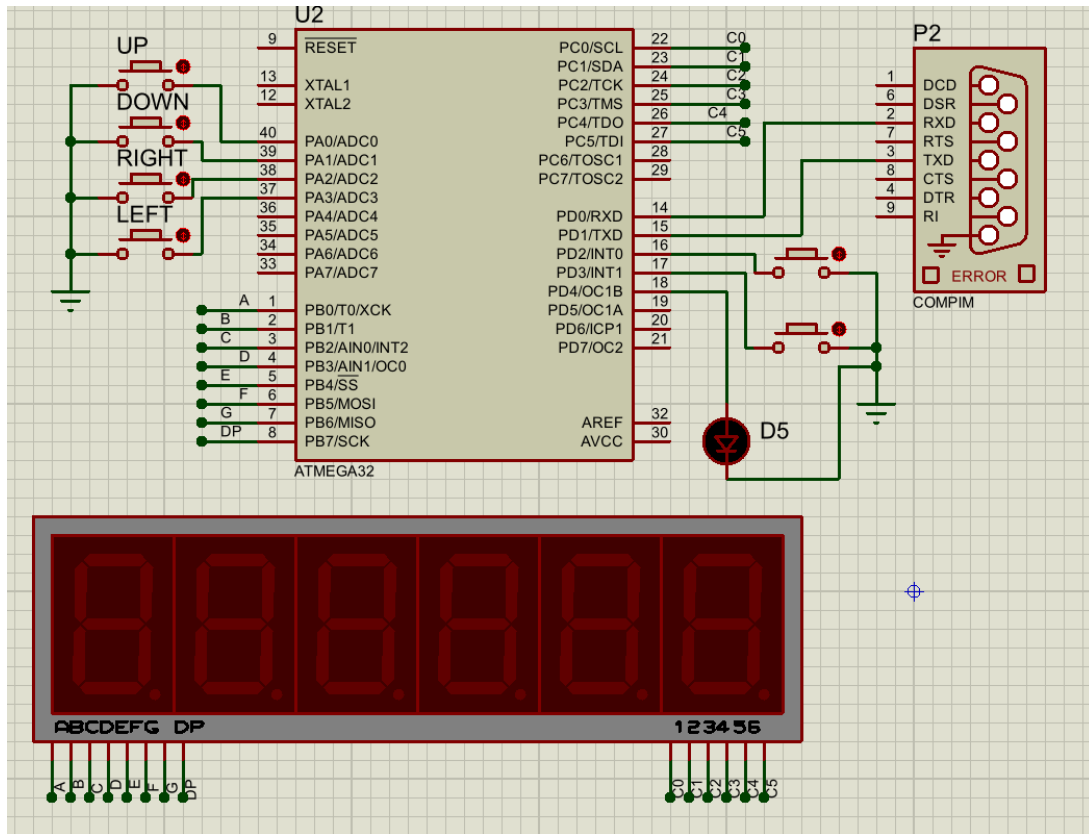


Figure 3.1: Complete circuit schematic of the controller board.

The segment lines of the six-digit display are connected in parallel to Port B (PB0–PB7), with each segment passing through a current-limiting resistor. The six common-anode lines are driven by Port C (PC0–PC5); because the display is common-anode, the digit-enable lines are active-high and the segment data is active-low, which is reflected in the segment glyph table stored in firmware. The keypad is attached to PA0–PA3 as inputs with internal pull-ups enabled, so a pressed key reads as logic 0. The INT0 and INT1 push-buttons connect to PD2 and PD3 respectively, also relying on internal pull-ups and falling-edge detection. The UART lines TXD (PD0) and RXD (PD1) are wired across to the sensor board with a common ground. PD4 is configured as an output and drives the heartbeat LED through a series resistor.

Chapter 4

Software Design

4.1 Program Structure

The firmware is organized as a single C translation unit, with state held in a small number of `volatile` globals shared between the main loop and the interrupt service routines. The structure can be summarized as:

- **Glyph table** (`seg_code[]`) — segment patterns for digits 0–9 plus blank, minus, and the letters C, F, H.
- **Clock state** — `hours`, `minutes`, `seconds`.
- **Multiplexing state** — `display_buf[6]` and `current_digit`.
- **Screen/edit state** — `mode`, `clock_edit`, `cursor` (0 = hours, 1 = minutes, 2 = seconds), and the `blink[6]` array.
- **Sensor cache** — `current_temp` plus the `temp_request` flag.
- **ISRs** — INT0 (screen cycle), INT1 (edit toggle), TIM1_COMPA (one-second timekeeping), TIM2_COMP (2 ms multiplexing, keypad, blink, heartbeat), USART_RX (sensor reply).
- **Helpers** — `uart_send`, `poll_temperature`, `apply_keys`, `scan_edit_keys`, `set_blink(i, on)`, `show_time`, `draw_env_value`, `show_temperature`, `show_fahr`, `show_humidity`, `update_display`.
- **main()** — initializes all ports, timers, peripherals, and interrupts, then runs the super-loop (`poll_temperature + update_display`).

4.2 Timer Configuration

Two timers are used in CTC (Clear-Timer-on-Compare-Match) mode.

Timer1 (16-bit) provides the one-second timekeeping base. With prescaler /256 and OCR1A set for 31250 counts:

$$f_{T1} = \frac{8 \times 10^6}{256 \times 31250} = 1 \text{ Hz} \quad (4.1)$$

The compare match fires once per second, driving the clock-update ISR.

Timer2 (8-bit) provides the 2 ms multiplexing cadence. With prescaler /64:

$$f_{T2} = \frac{8 \times 10^6}{64 \times (\text{OCR2} + 1)} = \frac{8 \times 10^6}{64 \times 250} = 500 \text{ Hz} \quad (4.2)$$

so the compare-match ISR fires exactly once every 2 ms. A private `heartbeat_tick` counter accumulates 500 such 2 ms ticks to produce the one-second heartbeat-LED toggle.

The relevant register settings are shown in Listing 4.1.

```

1  /* Timer1: CTC, prescaler /256, 1 s period */
2  TCCR1A = 0x00;
3  TCCR1B = (1<<WGM12) | (1<<CS12);
4  OCR1AH = 0x7A;
5  OCR1AL = 0x11;
6  TCNT1  = 0x0000;
7
8  /* Timer2: CTC, prescaler /64, 2 ms period */
9  TCCR2 = (1<<CTC2) | (1<<CS22);
10 OCR2  = 0xF9;
11 TCNT2 = 0x00;
12
13 TIMSK = (1<<OCIE1A) | (1<<OCIE2);

```

Listing 4.1: Timer1 and Timer2 initialization

4.3 Peripheral Configuration

I/O ports. Port A is an input port with internal pull-ups enabled on PA0–PA3 to interface the active-low keypad. Port B is a full output port driving the segment lines; its initial value is 0xFF (all segments off for common-anode). Port C uses PC0–PC5 as outputs for the six common-anode digit-enable lines. Port D configures PD0 (TXD) and PD4 (LED) as outputs, and PD2/PD3 (INT0/INT1) as inputs with pull-ups enabled for the falling-edge buttons.

USART. The USART is configured for 9600 baud, 8 data bits, no parity, one stop bit (8N1). With an 8 MHz clock the UBRR value is

$$\text{UBRR} = \frac{8 \times 10^6}{16 \times 9600} - 1 \approx 51.08 \rightarrow 51 \text{ (0x33)}, \quad (4.3)$$

giving a baud error of approximately 0.16%, well inside the $\pm 2\%$ tolerance of the receiver. Reception is interrupt-driven (RXCIE); the RX interrupt is re-enabled on every poll cycle. Transmission is polled in `uart_send()` by waiting on the UDRE flag.

Unused peripherals. To minimize power and avoid spurious activity, the analogue comparator (ACSR/ACD), ADC, SPI, TWI, and Timer0 are all explicitly disabled at start-up.

4.4 Interrupt Architecture

Five interrupt sources are configured, with the hardware priority order shown in Table 4.1 (determined by the ATmega32 interrupt vector table – lower vector address = higher priority).

Table 4.1: Interrupt vector table and priority

Priority	Vector	Interrupt Source	Function
1	0x002	RESET	Power-up / watchdog reset
2	0x004	EXT_INT0	Screen cycle (mode switching)
3	0x006	EXT_INT1	Edit-mode toggle
4	0x014	TIM1_COMPA	One-second timekeeping
5	0x016	TIM2_COMP	2 ms multiplexing heartbeat
6	0x01C	USART_RXC	Sensor data reception

4.4.1 INT0 — Screen Cycle

Triggered on the falling edge of PD2. Increments `mode` with wrap-around at `MODE_HUM`. This ISR is intentionally minimal: it modifies only a single `volatile` byte and returns.

4.4.2 INT1 — Edit Mode Toggle

Triggered on the falling edge of PD3. Toggles the `clock_edit` flag. When `clock_edit` transitions from 0 to 1, the system enters edit mode and the cursor defaults to the hours field. When it transitions back to 0, editing ends and the blink array is cleared.

4.4.3 TIM1_COMPA — One-Second Timekeeping

The highest-priority timer interrupt, firing exactly once per second. Increments `seconds`; on reaching 60, resets seconds, sets `temp_request`, and cascades to minutes (wrap at 60) and hours (wrap at 24).

4.4.4 TIM2_COMP — 2 ms Multiplexing Heartbeat

The busiest ISR, firing every 2 ms. It performs the following in sequence:

1. Call `scan_edit_keys()` to detect keypad events.
2. Turn off all digits to prevent ghosting.
3. Determine whether the current digit should be displayed or blanked (edit-cursor blink).
4. Output the segment code to `PORTB`.
5. Apply colon decimal points for TIME mode.
6. Enable the current digit via `PORTC`.
7. Advance the digit counter.
8. Increment the heartbeat tick counter; toggle PD4 every 500 ticks.

4.4.5 USART_RXC — Sensor Data Reception

Triggered when a byte is received on the UART. Stores the received byte from `UDR` into `current_temp`. Because a single-byte store on the AVR architecture is atomic, no critical section is required.

4.5 Display Buffer Mechanism

The display system uses a double-buffering approach via the `display_buf[]` array:

- `display_buf[6]` is a `volatile unsigned char` array indexed by digit position (0 = leftmost, 5 = rightmost).
- Each element stores an index into the `seg_code[]` glyph table.
- The **main loop** writes to `display_buf[]` via `update_display()`, which fills the buffer based on the active screen mode.
- The **Timer2 ISR** reads from `display_buf[]` sequentially, one digit per 2 ms tick, and outputs the corresponding segment code to `PORTB`.

This separation ensures that the display refresh is never blocked by the main loop's operations. Even if `update_display()` takes a variable amount of time (e.g., due to conditional branches for different modes), the multiplexing ISR continues to run at a precise 2 ms interval.

The `blink[6]` array works as an overlay: when `blink[i]` is set and bit 6 of the global `tick` counter is 1, digit *i* is blanked instead of shown. This produces the 2 Hz edit-cursor blink without modifying the display-buffer contents.

4.6 Keypad Debouncing

Mechanical push-buttons and keypad switches exhibit contact bounce: when pressed or released, the electrical contact oscillates between open and closed states for several milliseconds before settling. Without debouncing, a single press could be interpreted as multiple key events.

The firmware implements software debouncing via the `scan_edit_keys()` function, which runs inside the Timer2 ISR every 2 ms. The algorithm is:

1. Read the current state of Port A pins PA0–PA3 (`curr = PINA & 0x0F`).
2. Maintain a `prev` variable holding the previous sample.
3. Compute falling edges: `falling = (prev ^ curr) & prev`. This detects transitions from HIGH (released) to LOW (pressed).
4. Update `prev = curr` for the next iteration.
5. If any falling edge is detected, call `apply_keys(falling)` immediately.

This is effectively a periodic-sampling debouncer with a 2 ms sampling interval. Because the keypad is sampled at 500 Hz, the typical contact-bounce duration (5–20 ms) spans multiple samples, but only the first sample after the transition triggers the action; subsequent samples within the same press show no new falling edge. This rejects both the initial bounce train and any subsequent noise events shorter than the sampling period.

4.7 Global Variables

All variables shared between the main loop and interrupt service routines are declared with the `volatile` qualifier. This prevents the compiler from optimizing away reads or writes that an ISR may change asynchronously.

Table 4.2 lists the global variables used in the firmware.

Table 4.2: Global variable declarations and purpose

Variable	Type	Purpose
<code>seg_code[]</code>	<code>unsigned char[15]</code>	Segment glyph lookup table (read-only)
<code>display_buf[6]</code>	<code>volatile unsigned char[6]</code>	Glyph indices for each digit position
<code>current_digit</code>	<code>volatile unsigned char</code>	Currently active digit (0–5)
<code>mode</code>	<code>volatile unsigned char</code>	Active screen: TIME, TEMP, FAHR, or HUM
<code>clock_edit</code>	<code>volatile unsigned char</code>	Edit-mode flag (0 = off, 1 = on)
<code>cursor</code>	<code>volatile unsigned char</code>	Edit cursor: 0 = hours, 1 = minutes, 2 = seconds
<code>hours</code>	<code>volatile unsigned char</code>	Clock hours (0–23)
<code>minutes</code>	<code>volatile unsigned char</code>	Clock minutes (0–59)
<code>seconds</code>	<code>volatile unsigned char</code>	Clock seconds (0–59)
<code>temp_request</code>	<code>volatile unsigned char</code>	Flag set once per minute to trigger UART poll
<code>current_temp</code>	<code>volatile signed char</code>	Last received sensor value
<code>blink[6]</code>	<code>volatile uint8_t[6]</code>	Per-digit blink enable for edit cursor
<code>tick</code>	<code>volatile uint16_t</code>	Free-running counter for blink timing

On the AVR architecture, single-byte reads and writes are atomic. The variables `seconds`, `minutes`, `hours`, `mode`, `clock_edit`, `cursor`, `temp_request`, and `current_temp` are all single bytes, so no critical section is needed when accessing them from both contexts. The 16-bit `tick` variable is accessed only inside the Timer2 ISR, so it does not require synchronization either.

Chapter 5

Implementation

5.1 Development Environment

The firmware is developed in C using CodeVisionAVR, which provides the `<mega32.h>` header, the `interrupt[...]` ISR syntax, and the `#asm("sei")` intrinsic used to enable global interrupts. The project is configured for an ATmega32 target clocked at 8 MHz. Functional verification is carried out in Proteus Design Suite, using the project file `SysDig2AVRProject.pdsprj`, in which the display/controller ATmega32 is loaded with the compiled `final.hex` image and the companion sensor board is loaded with `SysDig2AVRProject.hex`.

5.2 Main Program

The `main()` function performs the entire initialization phase and then enters an infinite super-loop. Initialization sets the port data-direction registers and pull-ups, configures Timer1 and Timer2, the external interrupts, and the USART, and disables the unused peripherals. After global interrupts are enabled, the super-loop repeatedly calls `poll_temperature()` (which issues a one-byte UART request once per minute) and `update_display()` (which regenerates the six-digit buffer for the active screen and manages the blink array).

5.3 Key Implementation Details

5.3.1 Timer1 Compare ISR (One-Second Timekeeping)

```
1 interrupt [TIM1_COMPA] void timer1_compa_isr(void)
2 {
3     if (++seconds >= 60)
4     {
5         seconds = 0;
6         temp_request = 1;
7         if (++minutes >= 60)
8         {
9             minutes = 0;
10            if (++hours >= 24) hours = 0;
11        }
12    }
13 }
```

Listing 5.1: One-second timekeeping ISR

5.3.2 Timer2 Compare ISR (2 ms Multiplexing Heartbeat)

The 2 ms heartbeat simultaneously drives the keypad scan, the digit multiplexing, the colon and edit-cursor blink, and toggles the heartbeat LED every 500 ticks.

```

1 interrupt [TIM2_COMP] void timer2_comp_isr(void)
2 {
3     static unsigned int heartbeat_tick = 0;
4
5     scan_edit_keys();
6
7     PORTC = 0x00;           /* all digits off */
8
9     unsigned char show = 1;
10    if (blink[current_digit] && ((tick >> 6) & 1))
11        show = 0;
12    tick++;
13
14    if (show)
15        PORTB = seg_code[display_buf[current_digit]];
16    else
17        PORTB = seg_code[BLANK];
18
19    /* colon on digit 2 (HH:MM) and digit 4 (MM:SS),
20     blinks once per second with seconds parity */
21    if (mode == MODE_TIME)
22        if ((current_digit == 2 || current_digit == 4) && (seconds & 1))
23            PORTB &= 0x7F; /* DP ON (common anode) */
24
25    PORTC = (1 << current_digit);
26    if (++current_digit >= NUM_DIGITS) current_digit = 0;
27
28    if (++heartbeat_tick >= 500)
29    {
30        heartbeat_tick = 0;
31        PORTD ^= 0x10;      /* heartbeat LED toggle */
32    }
33 }

```

Listing 5.2: Multiplexing and heartbeat ISR

5.3.3 Edit-Cursor Blink Mapping

The cursor position selects which digit pair blinks: `cursor=0` (hours) blinks digits 0–1, `cursor=1` (minutes) blinks digits 2–3, `cursor=2` (seconds) blinks digits 4–5. The blink is controlled on a per-digit basis through `set_blink(i, on)`.

```

1 void set_blink(unsigned char i, unsigned char on)
2 {
3     blink[i] = on;
4 }
5
6 void update_display(void)
7 {
8     unsigned char i;
9     for (i = 0; i < NUM_DIGITS; i++) set_blink(i, 0); // always start
10    clean
11
12    switch (mode)
13    {

```

```

13     case MODE_TIME:
14         if (clock_edit)
15         {
16             if (cursor == 0)      { set_blink(0, 1); set_blink(1, 1)
17                                     ; }
18             else if (cursor == 1) { set_blink(2, 1); set_blink(3, 1)
19                                     ; }
20             else                  { set_blink(4, 1); set_blink(5, 1)
21                                     ; }
22         }
23         show_time();
24         break;
25
26     case MODE_TEMP: show_temperature(); break;
27     case MODE_FAHR: show_fahr();        break;
28     case MODE_HUM:  show_humidity();    break;
29 }

```

Listing 5.3: Update-display and blink mapping

5.3.4 Environmental Value Formatting

The `draw_env_value` function right-justifies the signed value in digits 0–2 with a leading blank and an optional minus sign. The unit letter (C, F, or H) appears in the last digit. For Celsius and Fahrenheit modes, a separator glyph is shown in digit 4; for humidity, digits 3–4 remain blank.

```

1 void draw_env_value(signed char val, unsigned char unit)
2 {
3     if (val < 0)
4     {
5         display_buf[0] = MINUS;
6         val = -val;
7     }
8     else
9         display_buf[0] = (val >= 100) ? (val / 100) : BLANK;
10
11     if (val >= 100)
12     {
13         display_buf[1] = (val / 10) % 10;
14         display_buf[2] = val % 10;
15     }
16     else if (val >= 10)
17     {
18         display_buf[1] = val / 10;
19         display_buf[2] = val % 10;
20     }
21     else
22     {
23         display_buf[1] = BLANK;
24         display_buf[2] = val;
25     }
26
27     display_buf[3] = BLANK;
28     display_buf[4] = BLANK;
29     display_buf[5] = unit;
30 }

```

Listing 5.4: Environmental value formatting**5.3.5 Keypad Editing of Hours/Minutes/Seconds**

```
1 void apply_keys(unsigned char pressed)
2 {
3     if (!clock_edit) return;
4
5     if (pressed & 0x04) { if (cursor < 2) cursor++; } /* RIGHT */
6     if (pressed & 0x08) { if (cursor > 0) cursor--; } /* LEFT */
7
8     if (pressed & 0x01) { /* UP */
9         if (cursor == 0) { if (++hours >= 24) hours = 0; }
10        else if (cursor == 1) { if (++minutes >= 60) minutes = 0; }
11        else { if (++seconds >= 60) seconds = 0; }
12    }
13    if (pressed & 0x02) { /* DOWN */
14        if (cursor == 0) { if (hours == 0) hours = 23; else
15            hours--; }
16        else if (cursor == 1) { if (minutes == 0) minutes = 59; else
17            minutes--; }
18        else { if (seconds == 0) seconds = 59; else
19            seconds--; }
20    }
21 }
```

Listing 5.5: Keypad handling for time editing

Chapter 6

Timing Analysis

6.1 Timer1 Timing Diagram

Timer1 generates a precise one-second interrupt using CTC mode. With an 8 MHz system clock, a prescaler of /256, and OCR1A = 31250, the compare match fires at exactly 1 Hz. Figure 6.1 illustrates the Timer1 timing.

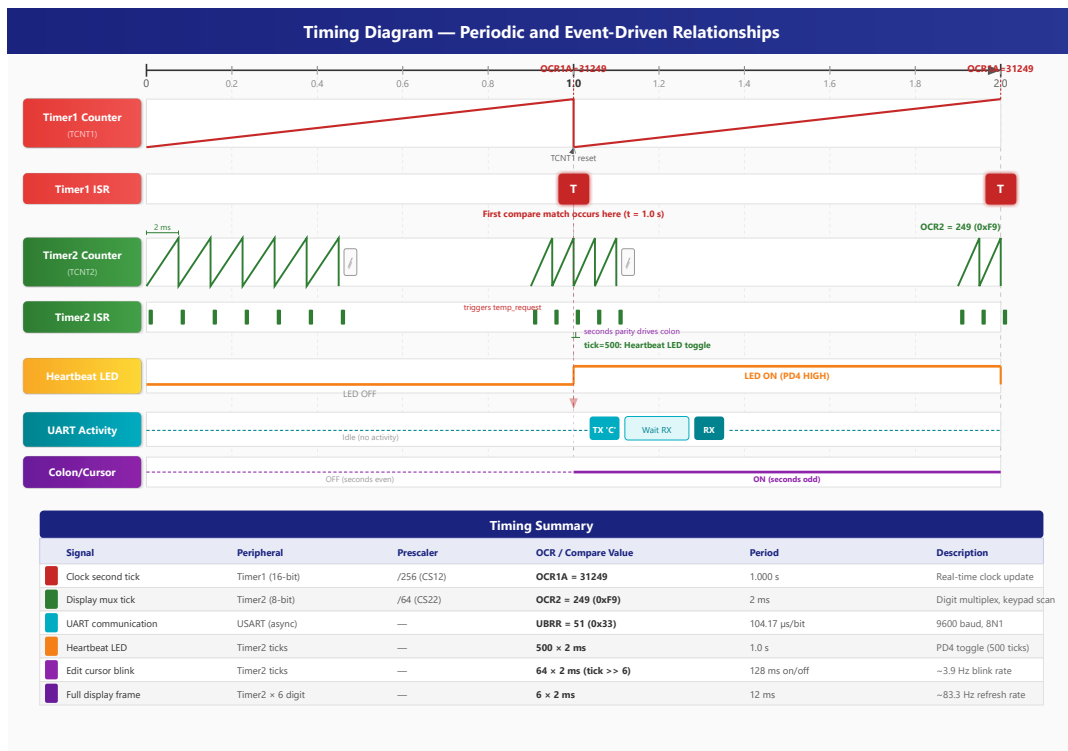


Figure 6.1: Timing Diagram

The timer counts from 0 to 31250, taking:

$$t_{\text{count}} = \frac{31250}{8 \times 10^6 / 256} = \frac{31250}{31250} = 1 \text{ s} \quad (6.1)$$

6.2 Timer2 Multiplexing Timing

Timer2 fires at 500 Hz (2 ms period) and drives the six-digit multiplexing sequence:

- **Digit ON time:** 2 ms per digit.
- **Frame time:** $6 \times 2 \text{ ms} = 12 \text{ ms}$.
- **Refresh rate:** $1/12 \text{ ms} \approx 83.3 \text{ Hz}$.

The blink timing uses the `tick` counter, incremented every 2 ms ISR:

- Blink pattern toggles every 64 ticks: $2 \text{ ms} \times 64 = 128 \text{ ms}$.
- Full blink period: $2 \times 128 \text{ ms} = 256 \text{ ms}$.
- Blink frequency: $1/256 \text{ ms} \approx 3.9 \text{ Hz}$.

The heartbeat LED toggles every 500 ticks:

$$t_{\text{heartbeat}} = 2 \text{ ms} \times 500 = 1 \text{ s} \quad (6.2)$$

6.3 UART Communication Timing

The UART operates at 9600 baud (8N1). At 9600 baud, each bit has a duration of:

$$t_{\text{bit}} = \frac{1}{9600} \approx 104.17 \text{ } \mu\text{s} \quad (6.3)$$

A complete 10-bit frame (1 start, 8 data, 1 stop) takes:

$$t_{\text{frame}} = 10 \times 104.17 \text{ } \mu\text{s} \approx 1.042 \text{ ms} \quad (6.4)$$

The sensor request/response transaction involves:

1. ATmega32 transmits one request byte (1.042 ms).
2. Sensor board processes and responds (bounded by sensor firmware).
3. Sensor board transmits one response byte (1.042 ms).

The total transaction time is approximately 2.1 ms plus sensor processing latency, well within the 60-second polling interval.

Chapter 7

Testing and Results

7.1 Simulation Environment

The design was validated using the following tools:

- **CodeVisionAVR** — C compiler and IDE for firmware development.
- **Proteus Design Suite** — Circuit simulation and microcontroller emulation.
- **Clock frequency** — 8 MHz (system clock).
- **Display model** — 7SEG-MPX6-CA (6-digit common-anode).

7.2 Simulation Results

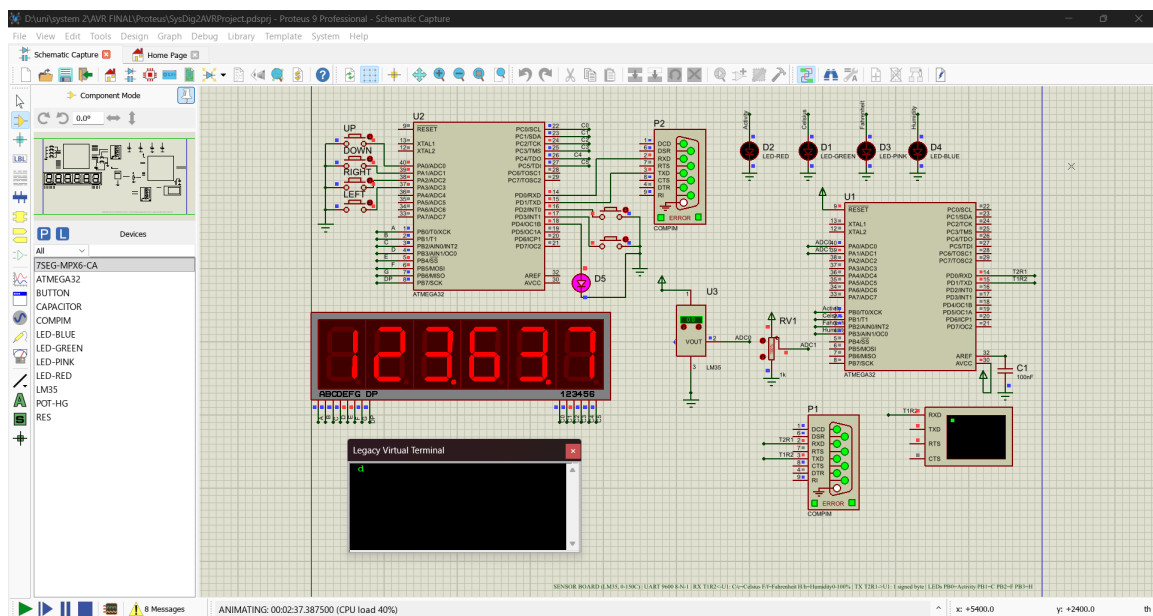


Figure 7.1: Proteus simulation screenshot showing the ATmega32 driving the 7SEG-MPX6-CA display.

The simulation confirmed:

- The TIME screen shows 12:34:00 at start-up, then increments seconds at the correct 1 Hz rate; the two colons (between HH:MM and MM:SS) blink once per second in step with the seconds parity.

- The heartbeat LED on PD4 toggles once per second.
- Pressing INTO cycles the display through TIME, TEMP, FAHR, and HUM, and the unit letter (C, F, or H) appears in the last (sixth) digit of the corresponding sensor screen, with the numeric value right-justified in digits 0–2 and the separator glyph in digit 4 for temperature screens.
- On the TIME screen, INT1 enters edit mode; the LEFT/RIGHT keys move the cursor among the hours, minutes, and seconds fields, and the digit pair of the selected field blinks at approximately 2 Hz while the rest of the clock keeps counting.
- UP/DOWN adjust the selected field with wrap-around (hours modulo 24, minutes/seconds modulo 60); the clock keeps counting in the background.
- After 60 s on a sensor screen, a new UART request is issued and the displayed value refreshes.

7.3 UART Manual Reception Verification

To confirm that the USART receive path correctly updates the internal `current_temp` variable and that the display logic renders it exactly as intended, the sensor board was bypassed and single bytes were injected manually through a virtual COM port using the Serial Port Utility, simulating the sensor's reply for each mode.

7.3.1 Celsius Mode

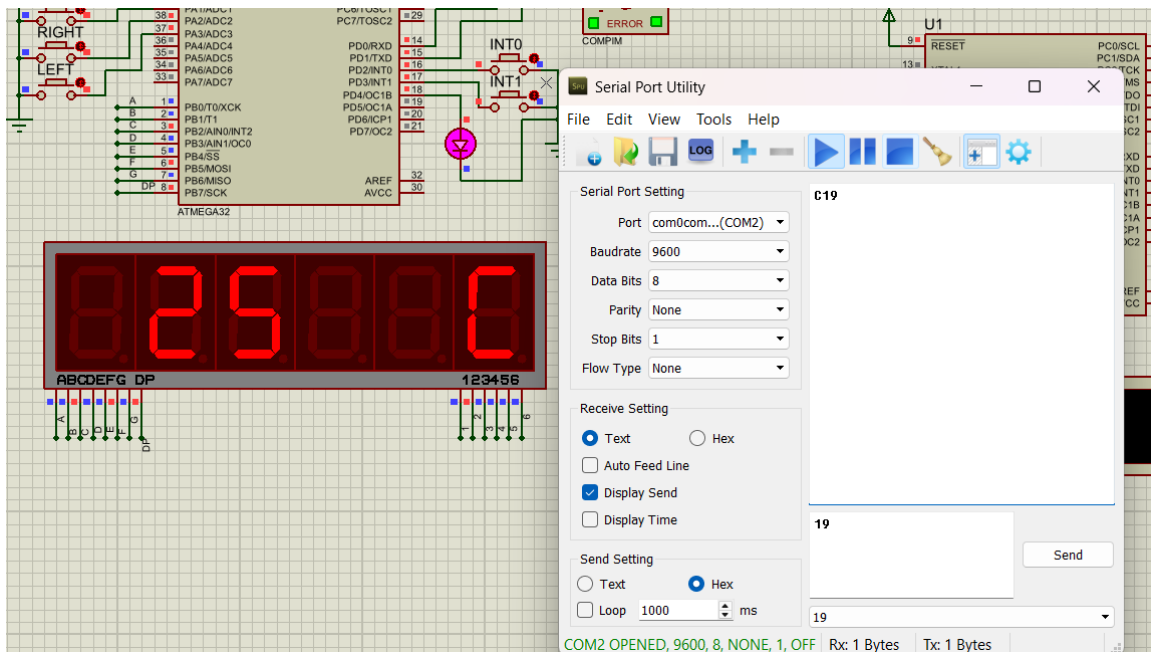


Figure 7.2: Manual UART test in Celsius mode: byte 0x19 (25 decimal) sent in response to the 'C' request.

A request byte 'C' was observed being issued by the board, and a reply of 0x19 (25 in decimal) was sent back manually. The display showed 25 right-justified in digits 1–2 with the C unit glyph in digit 5, matching exactly what `draw_env_value(25, CHAR_C)` computes. The result matched the expected output.

7.3.2 Fahrenheit Mode

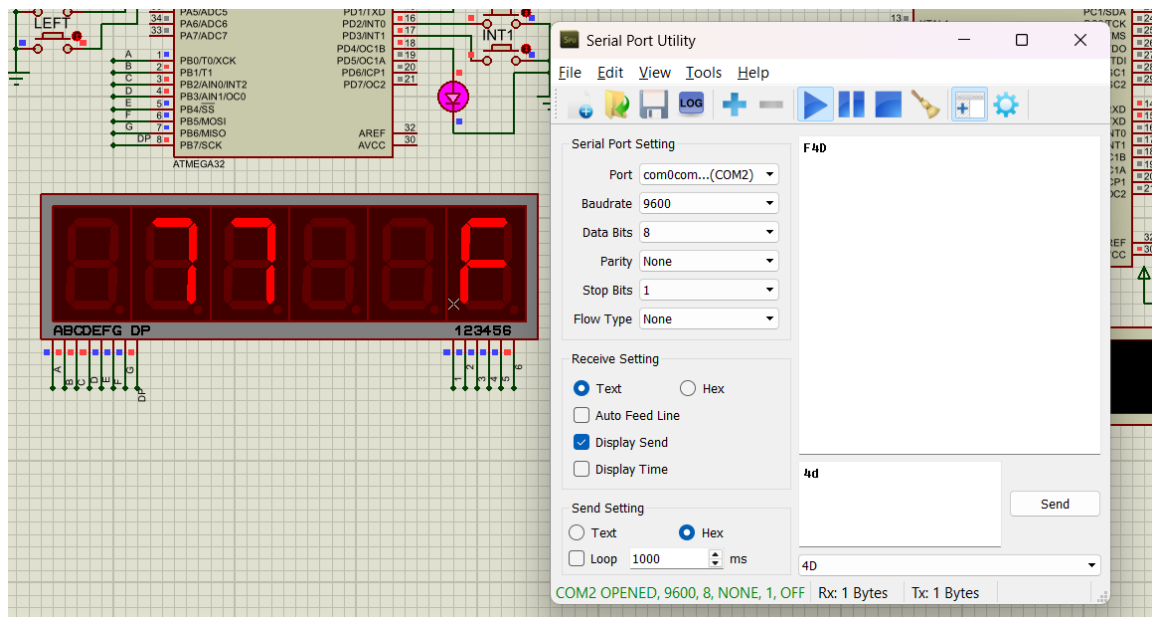


Figure 7.3: Manual UART test in Fahrenheit mode: byte 0x4D (77 decimal) sent in response to the 'F' request.

A request byte 'F' was issued by the board, and a reply of 0x4D (77 decimal, the Fahrenheit equivalent of 25°C) was sent back. The display showed 77 with the F unit glyph, again matching the expected formatting. The result matched the expected output.

7.3.3 Humidity Mode

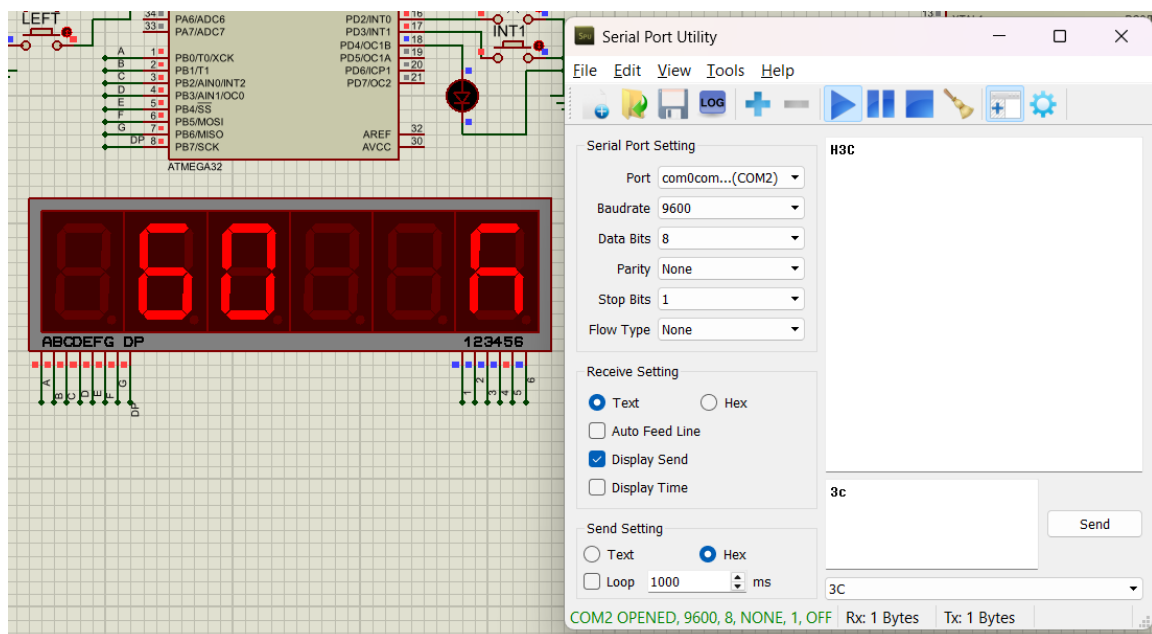


Figure 7.4: Manual UART test in Humidity mode: byte 0x3C (60 decimal) sent in response to the 'H' request.

A request byte 'H' was issued, and a reply of 0x3C (60 decimal) was sent back, representing 60% relative humidity. The display showed 60 with the H unit glyph in digit 5. The result matched

the expected output.

7.3.4 Negative Value Handling

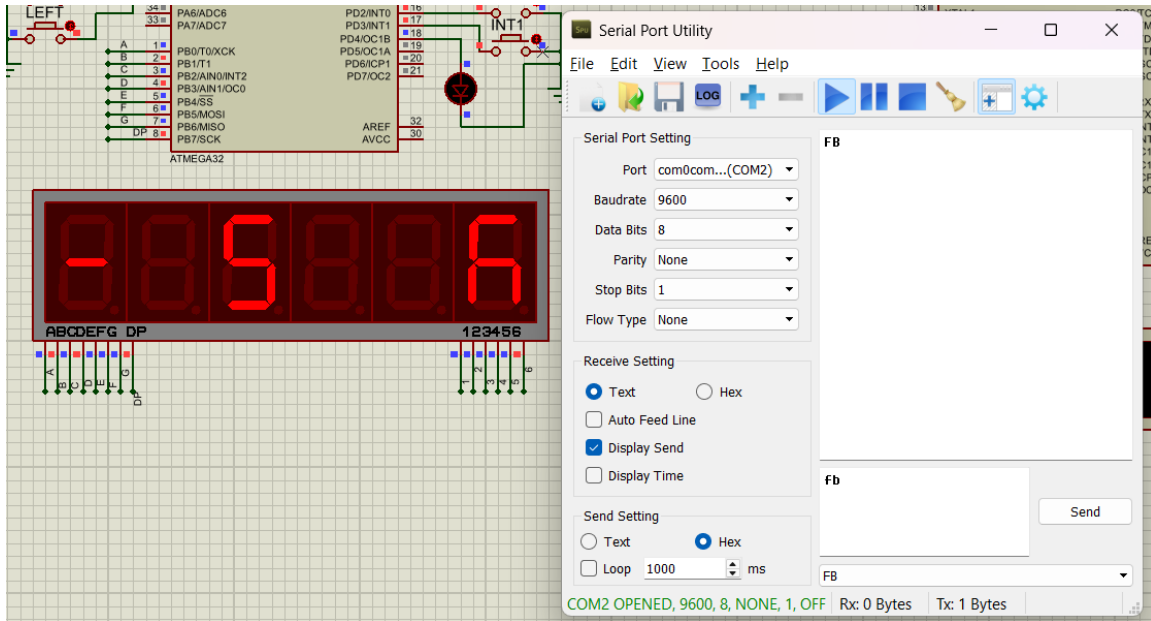
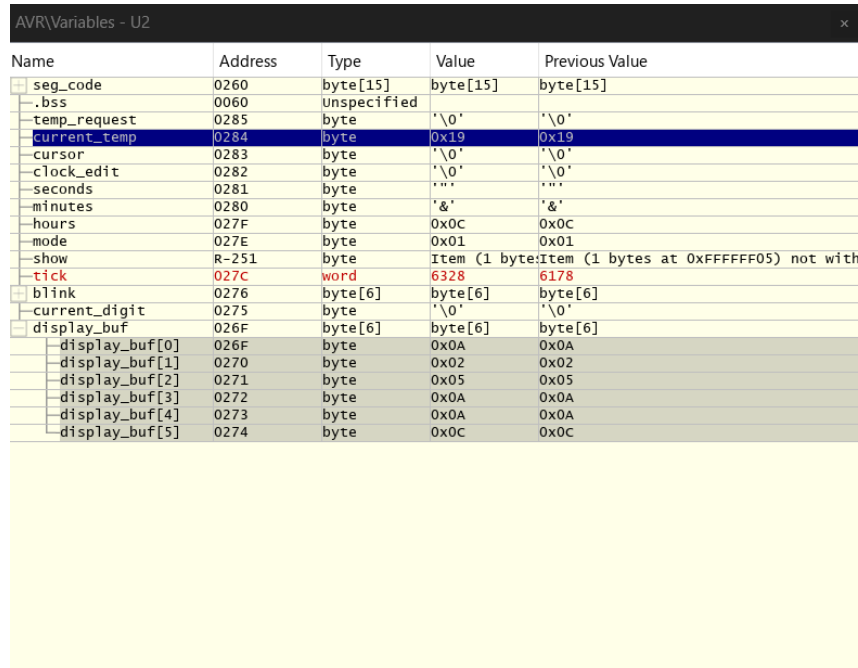


Figure 7.5: Manual UART test with a negative reading: byte 0xFB (two's-complement representation of -5) sent while in Humidity mode.

To verify correct handling of negative **signed char** values, a reply byte of 0xFB (the two's-complement encoding of -5) was sent. The display correctly showed a minus sign in digit 0 followed by 5 in digit 2, with the unit glyph unaffected, confirming that the sign-and-magnitude branch of `draw_env_value()` operates correctly for out-of-range or below-zero sensor readings. The result matched the expected output.

7.3.5 Direct Variable Inspection



Name	Address	Type	Value	Previous Value
seg_code	0260	byte[15]	byte[15]	byte[15]
.bss	0060	unspecified		
temp_request	0285	byte	'\0'	'\0'
current_temp	0284	byte	0x19	0x19
cursor	0283	byte	'\0'	'\0'
clock_edit	0282	byte	'\0'	'\0'
seconds	0281	byte	''	''
minutes	0280	byte	'&'	'&'
hours	027F	byte	0x0C	0x0C
mode	027E	byte	0x01	0x01
show	R-251	byte	Item (1 byte:Item (1 bytes at 0xFFFFF05) not with	
tick	027C	word	6328	6178
blink	0276	byte[6]	byte[6]	byte[6]
current_digit	0275	byte	'\0'	'\0'
display_buf	026F	byte[6]	byte[6]	byte[6]
display_buf[0]	026F	byte	0x0A	0x0A
display_buf[1]	0270	byte	0x02	0x02
display_buf[2]	0271	byte	0x05	0x05
display_buf[3]	0272	byte	0x0A	0x0A
display_buf[4]	0273	byte	0x0A	0x0A
display_buf[5]	0274	byte	0x0C	0x0C

Figure 7.6: Proteus Variables window confirming `current_temp` holds the exact byte received (0x19), with `display_buf` correctly populated as 0x0A, 0x02, 0x05, 0x0A, 0x0A, 0x0C.

To rule out any ambiguity introduced by the multiplexed display refresh (where a single still frame can only show one active digit at a time), the internal `current_temp` and `display_buf` variables were inspected directly in Proteus’s memory/variable viewer immediately after the Celsius test. `current_temp` held 0x19 (25 decimal), and `display_buf` held {0x0A, 0x02, 0x05, 0x0A, 0x0A, 0x0C} — an exact match to BLANK, 2, 5, BLANK, BLANK, CHAR_C. This confirms, independent of the physical display rendering, that the UART reception and buffer computation are functioning correctly end to end.

Table 7.1: Summary of manual UART injection test cases

Test Case	Byte Sent	Expected Result	Observed
Celsius reply	0x19 (25)	25 with C glyph	Confirmed
Fahrenheit re- ply	0x4D (77)	77 with F glyph	Confirmed
Humidity reply	0x3C (60)	60 with H glyph	Confirmed
Negative reply	0xFB (-5)	Minus sign, 5, unit glyph	Confirmed
Variable check	0x19 (25)	<code>current_temp == 0x19</code>	Confirmed

7.4 Experimental Results

Table 7.2: Summary of functional test cases and observed results

Test Case	Expected Result	Observed
Power-on display	TIME screen showing 12:34:00	Confirmed
INT0 cycling	TIME→TEMP→FAHR→HUM→TIME	Confirmed
Colon blink rate	Once per second on both colons	Confirmed
Heartbeat LED	PD4 toggles every 1 s	Confirmed
Edit-mode entry	Press INT1 on TIME	Cursor on hours blinks at ≈ 2 Hz
RIGHT/LEFT cursor	Moves H→M→S and back	Confirmed
UP/DOWN adjust	Wrap at 24/60/60 for H/M/S	Confirmed
Clock while editing	Seconds keep counting	Confirmed
Sensor poll (C)	UART sends 'C', value updates	Confirmed
Sensor poll (F)	UART sends 'F', value updates	Confirmed
Sensor poll (H)	UART sends 'H', value updates	Confirmed
Sensor screen layout	Value in digits 0–2, unit in digit 5	Confirmed
Separator glyph	Shown in digit 4 for C/F modes	Confirmed

7.5 Debugging and Design Iterations

During development several issues were identified and corrected:

- **Clock keep-running during edit.** Earlier revisions risked freezing the clock while editing. The current design keeps the Timer1 one-second tick independent of the edit state, so the clock advances even while the user is adjusting fields.
- **Dual-timer separation.** Separating timekeeping (Timer1) from multiplexing (Timer2) eliminated jitter in the one-second tick that was present in a single-timer approach, where the multiplexing ISR had to accommodate both 2 ms and 1 s intervals.
- **Blink rate.** The blink comparison uses a shift of $\gg 6$ on the `tick` counter, which is incremented every 2 ms ISR. This toggles every $2 \text{ ms} \times 64 = 128 \text{ ms}$, giving a blink of $\approx 2 \text{ Hz}$ (256 ms period), which is readable and clearly indicates the field being edited.
- **Cursor wrap limits.** The cursor is constrained to $0 \leq \text{cursor} \leq 2$ by explicit checks in the LEFT/RIGHT handlers, preventing it from running off the end of the time fields.
- **Polling cadence.** Only the currently displayed sensor screen is refreshed each minute; switching modes mid-cycle means waiting up to one minute for a fresh reading. This is acceptable for the classroom scope but is noted as a limitation.

Chapter 8

Conclusion and Future Work

8.1 Conclusion

This project demonstrates the integration of timing, multiplexed display, external interrupts, keypad debouncing, a blinking edit cursor, and serial communication on a single ATmega32 microcontroller using two hardware timers. Timer1 in CTC mode provides a stable one-second timekeeping base for the 24-hour clock, while Timer2, running at a 2 ms cadence, provides a deterministic heartbeat for display multiplexing, keypad scanning, colon and cursor blinking, and the heartbeat LED. The six-digit display allows the full HH:MM:SS time format to be shown, and the sensor interface proves the viability of a distributed two-microcontroller architecture communicating over a single UART link with a simple request/response protocol. The edit-mode blinking cursor clearly indicates which field the user is adjusting, and the keep-running feature ensures the clock is never frozen during editing.

8.2 Future Work

- Measure temperature and humidity locally on the controller board using the on-chip ADC with a dedicated sensor, eliminating the separate sensor microcontroller.
- Persist the time in EEPROM so that a battery-backed or RTC-equipped design survives power cycles.
- Allow the user to adjust the polling rate.
- Use dedicated per-unit caches to avoid cross-unit contamination when switching between Celsius and Fahrenheit mid-cycle.
- Add a real-time clock IC for accurate long-term timekeeping.

Source Code Availability

The complete firmware source code (`main.c`) is provided separately as a supplementary file. This report includes only the key implementation sections required to explain the firmware architecture and operation, as presented in Chapter 5 and reproduced in Appendix A.

Bibliography

- [1] Atmel Corporation, “ATmega32 8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash,” Rev. 2503M–AVR–06/10, 2010. Available: https://ww1.microchip.com/downloads/en/DeviceDoc/Atmega32_Datasheet.pdf
- [2] Kingbright, “7SEG-MPX6-CA 0.56-Inch Six-Digit Common-Anode Seven-Segment Display Datasheet,” 2015.
- [3] P. Hansch, “CodeVisionAVR C Compiler Reference,” HP InfoTech, 2020.
- [4] Labcenter Electronics, “Proteus Design Suite User Manual,” 2023.
- [5] J. Maksmović and M. Babić, *Embedded Systems with AVR Microcontrollers: A Hands-On Approach*. New York: IEEE Press, 2018.
- [6] S. B. C. Štys, “Multiplexed seven-segment display decoding for embedded controllers,” *IEEE Transactions on Consumer Electronics*, vol. 64, no. 2, pp. 120–127, May 2018.
- [7] D. Gadre and A. M. Gosavi, *Programming and Customizing the AVR Microcontroller*. New York: McGraw-Hill, 2016.
- [8] “AVR C, Parts 1–4: Setup, GPIO, Bit Manipulation, and EEPROM,” Course Lecture Notes, Department of Electrical Engineering, Shahid Beheshti University.
- [9] “AVR C, Parts 5–6: AVR Hardware and Timer Programming,” Course Lecture Notes, Department of Electrical Engineering, Shahid Beheshti University.
- [10] “AVR C, Parts 7–8: Interrupts and Serial Port Programming,” Course Lecture Notes, Department of Electrical Engineering, Shahid Beheshti University.
- [11] M. A. Mazidi, S. Naimi, and S. Naimi, *The AVR Microcontroller and Embedded Systems: Using Assembly and C*. Upper Saddle River, NJ: Pearson Prentice Hall, 2011.

Appendix A

Firmware Listing Reference

The full firmware, `main.c`, is distributed as a supplementary file alongside this report and is not reproduced in full here. Section 5 presents the ISR and helper-function excerpts (Listings 4.1–5.5) that are essential to understanding the multiplexing, timekeeping, editing, and UART-formatting logic. Readers requiring the complete, compilable source should refer to the supplementary `main.c` file submitted with this report.